FPGA-Based Intelligent Elevator Control System

EEE F314 : FPGA Based System Design



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

HYDERABAD CAMPUS

Name	ID
Y SAMEEKSHA	2023A3PS0367H
KARTIKEYA SRIVASTAVA	2023A3PS0385H
RITHUL C	2023A3PS0391H
DUVVURI SRAVAN	2023A8PS0438H
AARUSH KALRA	2023AAPS1120H

Demonstration Video Link - https://youtu.be/J0O-CjP-Rik?si=Xs1sV8EZXlBVCWqD

1. Abstract

This project presents the design and implementation of an Intelligent Elevator Control System that utilizes an FPGA for the control of an 8-floor vertical transportation unit. The core objective was to move beyond simple sequential control to an advanced, real-time scheduling logic capable of handling concurrent and dynamic floor requests efficiently and safely. The intelligence of the system emanates from the fact that it is totally synchronous, hierarchical, and time-based, realized through Verilog Hardware Description Language (HDL).

The main architectural elements are a top-level module-elevator, high-precision clock divider and the central core Finite State Machine. Some of the inherent benefits from choosing an FPGA come through massive parallelism and deterministic execution times, crucial for safety-critical systems like elevators. This project addresses a common challenge in traditional micro-controller-based systems: the non-deterministic latency in processing urgent requests. By implementing the control logic directly in hardware, the system ensures reliable and rapid response times.

A key innovation in the hardware-software interaction is the treatment of input signals. Since physical slide switches are used to simulate momentary floor call buttons, the top module incorporates dedicated digital logic for rising-edge detection. This converts a persistent switch input into a single-cycle request pulse, ensuring that a call is added to the service queue only once per activation, preventing redundant or runaway requests. This one-shot pulse then feeds into the FSM core, which operates on a strictly defined, real-time clock enable signal derived from the clock divider.

The clock divider module is essential for simulating real-world time constraints, dividing the 100 MHz system clock down to a 1 Hz pulse. All critical state transitions—movement between floors, door operation, and idle timeout are synchronized to this 1 Hz rate, effectively equating one clock tick to one second of simulated time. The central FSM implements a sequential request service queue that stores up to 8 unique floor requests in a latch register. The state machine transitions between S_IDLE, S_MOVING_UP, S_MOVING_DOWN, and S_DOOR_OPEN, prioritizing requests based on a simple first-come, first-served mechanism combined with safety checks. A dedicated timer is implemented for door open duration, ensuring passengers have adequate time to board and alight, and an idle timeout mechanism is included to return the elevator to the ground floor (Floor 0) after a period of inactivity, maximizing energy efficiency and readiness for new traffic. Furthermore, a simple overload condition is monitored, triggering a dedicated LED indicator. The successful synthesis, simulation, and deployment of this design on the target FPGA platform demonstrate a robust and scalable solution for intelligent vertical transportation control. The final system maps all operational states, floor position, direction, and safety indicators to the on-board LEDs for comprehensive, real-time visual feedback.

2. Introduction

The efficient and safe movement of people and goods in multi-story buildings is governed by complex control systems. Modern elevator systems must be intelligent, capable of handling multiple, concurrent requests while optimizing travel time and minimizing energy consumption. Traditional hardwired relay logic is prone to failure and is inflexible, while purely software-based solutions can suffer from unpredictable latencies.

This project addresses these limitations by leveraging the power of FPGA technology. FPGAs offer high-speed, parallel processing capabilities, making them ideal for the deterministic, high-reliability control required in elevator operation. This report details the design of an 8-Floor Intelligent Elevator Control System implemented entirely in Verilog HDL. The system focuses on three pillars: Reliable Input Processing, Real-Time Simulation, and Intelligent State Management. By adopting a modular, synchronous design, the project successfully demonstrates a system that is not only functionally accurate but also fully testable and scalable for industrial application.

3. Objective

The primary objectives of the FPGA-Based Intelligent Elevator Control System project are:

- **Design and Implement an 8-Floor Control System:** To create a digital logic system capable of managing floor selection, movement, and door operation across 8 distinct floors (Floor 0 to Floor 7).
- **Establish Real-Time Simulation:** To implement a precision clock divider to synchronize the FSM operations (movement, door cycle) to a fixed, real-world time base (1 Hz, or 1 second per tick).
- **Implement Reliable Input Handling:** To design logic for rising-edge detection on switch inputs, converting continuous signals into single-pulse requests for reliable queueing.
- **Develop a State-Driven Control Core:** To design and implement a Finite State Machine (FSM) with states for `IDLE`, `MOVING_UP`, `MOVING_DOWN`, and `DOOR_OPEN` to manage elevator sequences.
- **Integrate Intelligent Features:** To include functionality for a sequential request queue, an idle timeout (return-to-ground functionality), and a simple overload safety indicator.
- **Verify on Target Hardware:** To map all inputs (Switches, Reset) and outputs (Floor Display, Direction, Door Status) to the LEDs and physical components of a target FPGA development board (ZEDBoard) via an XDC constraints file.

4. Methodology

4.1. System Architecture

The project employs a hierarchical, modular design structure to separate concerns and enhance testability. The overall architecture is composed of three main modules:

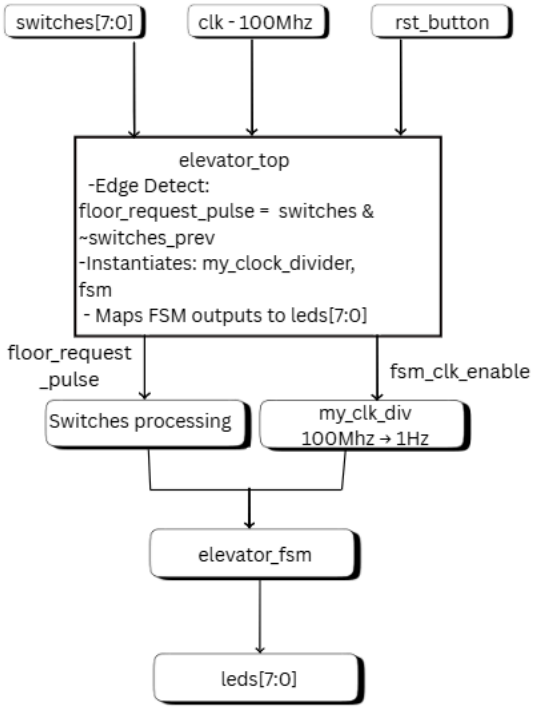
1. **`elevator_top` (Top-Level Wrapper):**
    a. **Function:** This module serves as the primary interface between the physical I/O ports of the FPGA (defined in the XDC file) and the core control logic. It handles all necessary port declarations and internal module instantiations.
    b. **Crucial Input Synchronization:** The top module is responsible for converting the physical slide switch inputs (switches[7:0]) into reliable, single-cycle request pulses. This is achieved via rising-edge detection logic. By registering the previous state of the switches (switches_prev) and combining this with the current state (switches & ~switches_prev), a one-tick signal (floor_request_pulse) is generated only when a switch is flipped ON. This prevents redundant requests from being queued while a switch is held in the ON position, simulating a momentary button press.
    c. **Mapping:** It instantiates the clock_divider and the elevator_fsm and connects the FSM outputs (current_floor, direction, door_open, Overload_LED) to the 8 user LEDs (leds[7:0]) for system visualization.
2. **`clock_divider` (Precision Clock Divider):**
    a. **Function:** This module is essential for implementing the real-time simulation aspect of the elevator control. It scales the high-frequency FPGA clock down to a human-perceptible, reliable time base.
    b. **Implementation:** Given the FPGA's 100 MHz input clock, the module utilizes a 27-bit synchronous counter (`counter_reg`) to count up to 100,000,000 - 1.
    c. **Output:** The output (clk_out, which becomes fsm_clk_enable) is asserted high for a single clock cycle (10 ns) precisely once every 100 million clock cycles, resulting in a 1 Hz pulse. This pulse is the synchronous heartbeat that gates all state updates and timing within the FSM.
3. **`elevator_fsm` (Finite State Machine Core):**
    a. **Function:** This is the heart of the intelligent control system. It manages the elevator's state, handles floor requests, implements timers, and determines the next movement and operation sequences.
    b. **Synchronization:** All state and register updates within this module are gated by the 1 Hz clk_enable pulse from the clock divider. This ensures that every movement step, every door cycle, and every timer increment correlates precisely to one second of simulated time.
    c. **State Logic:** The FSM implements a four-state model (S_IDLE, S_MOVING_UP, S_MOVING_DOWN, S_DOOR_OPEN) with complex transition logic that checks the sequential service queue (latch_requests) and the position of the elevator.

d. **Timing Control:** It contains dedicated, 1 Hz-driven counters: a move_timer (set to 2 cycles for 2 seconds of travel per floor), a door_timer (set to 10 cycles for 10 seconds of door-open time), and an idle_timer (set to 20 cycles for the 20-second return-to-ground feature).

4.2. Functional Block Diagram and Data Flow

The system operates as a closed-loop digital control system:

1. **Clock Input (`clk`):** The 100 MHz clock source enters the system.
2. **Clock Enable Generation:** The clock_divider counts up to 100 million cycles, generating a single-clock-cycle pulse (fsm_clk_enable) at a 1 Hz frequency. This pulse dictates the pace of the elevator simulation.
3. **Request Input & Synchronization:** The switches[7:0] inputs are processed by the Edge Detection Logic within the top module, comparing the current switch state with its previous state (switches & ~switches_prev). This creates a one-tick floor_request_pulse, ensuring a single request is registered regardless of how long the switch is held.
4. **FSM Core:** The elevator_fsm receives the 1 Hz clk_enable and the floor_request_pulse. State transitions and register updates (floor, direction, timers) occur only when clk_enable is high. The FSM manages the request queue and service logic.
5. **Output Mapping:** FSM outputs (current_floor, direction, door_open, Overload_LED) are mapped directly to the eight leds[7:0] on the ZEDBoard for visualization.



4.3. Working Principle

Finite State Machine (FSM)

The FSM operates on four primary states, synchronized by the 1 Hz clock enable pulse:

- **S_IDLE (2'b00):** The default state when no calls are pending.
    - **Transition to Moving:** If a valid request is in the queue, the FSM determines the required direction and transitions to `S_MOVING_UP` or `S_MOVING_DOWN`.
    - **Return to Ground:** If the elevator is not at Floor 0, and the `idle_timer` reaches `IDLE_TIMEOUT_CYCLES` (20 cycles, simulating 20 seconds), the FSM sets the target to Floor 0 and transitions to `S_MOVING_DOWN`.
- **S_MOVING_UP (2'b01) / S_MOVING_DOWN (2'b10):** The elevator is in transit.
    - **Movement Timer:** A `move_timer` counts up to `MOVE_CYCLES_PER_FLOOR` (2 cycles, simulating 2 seconds of travel) for each floor change.
    - **Floor Update:** When the `move_timer` resets, the `current_floor` register is incremented or decremented.
    - **Transition to Door Open:** If the `current_floor` matches a floor requested in the service queue, the FSM transitions to `S_DOOR_OPEN`.
- **S_DOOR_OPEN (2'b11):** The elevator has stopped at a requested floor.
    - **Door Timer:** A door_timer counts up to DOOR_OPEN_CYCLES (10 cycles, simulating 10 seconds).
    - **Request Servicing:** The corresponding bit in the request queue (latch_requests) is cleared.
    - **Transition to Idle/Moving:** Once the timer expires, the FSM checks the remaining queue. If the queue is empty, it returns to S_IDLE. Otherwise, it determines the next target and transitions to S_MOVING_UP or S_MOVING_DOWN.

Real-Time Clocking

The 100 MHz clock is divided by 10^8 to produce a 1 Hz pulse. This 1 Hz signal is used as a clock enable. All sequential logic within the FSM is sensitive to posedge clk but only updates when the clk_enable signal is high. This ensures precise, human-perceptible timing for all simulation elements: 2 seconds of travel between floors, 10 seconds of door open time, and a 20-second idle timeout.

4.4. Implementation Process

1. **HDL Development:** All modules were written in Verilog HDL.
2. **Simulation & Verification:** The (elevator_tb)testbench was used to simulate the FSM and verify core functionality, including the sequential servicing, movement timing, and return-to-ground feature.
3. **Synthesis and Implementation:** The Verilog design was synthesized and implemented using the Xilinx Vivado tool chain .

4. **Constraint Mapping:** The elevator_xdc file was applied to map the logical ports of the design to the physical pins of the target FPGA (ZEDBoard), connecting clk, rst_btn, switches, and leds to their respective hardware components.
5. **Bitstream Generation:** The final bitstream was generated and downloaded onto the FPGA for hardware verification.

5. Experimental Setup

5.1. Hardware Components / Software Tools Requirement

Type	Component	Purpose
Hardware	FPGA Development Board (ZEDBoard)	Target platform for running the synthesized design.
Hardware	On-board Switches (8)	Inputs for floor call requests (`switches[7:0]`).
Hardware	On-board LEDs (8)	Outputs for status display (`leds[7:0]`).
Hardware	On-board Push Button (1)	Input for system reset .
Software	Xilinx Vivado	Synthesis, Implementation, and Bitstream generation.
Software	Verilog HDL	Description language for the digital logic design.

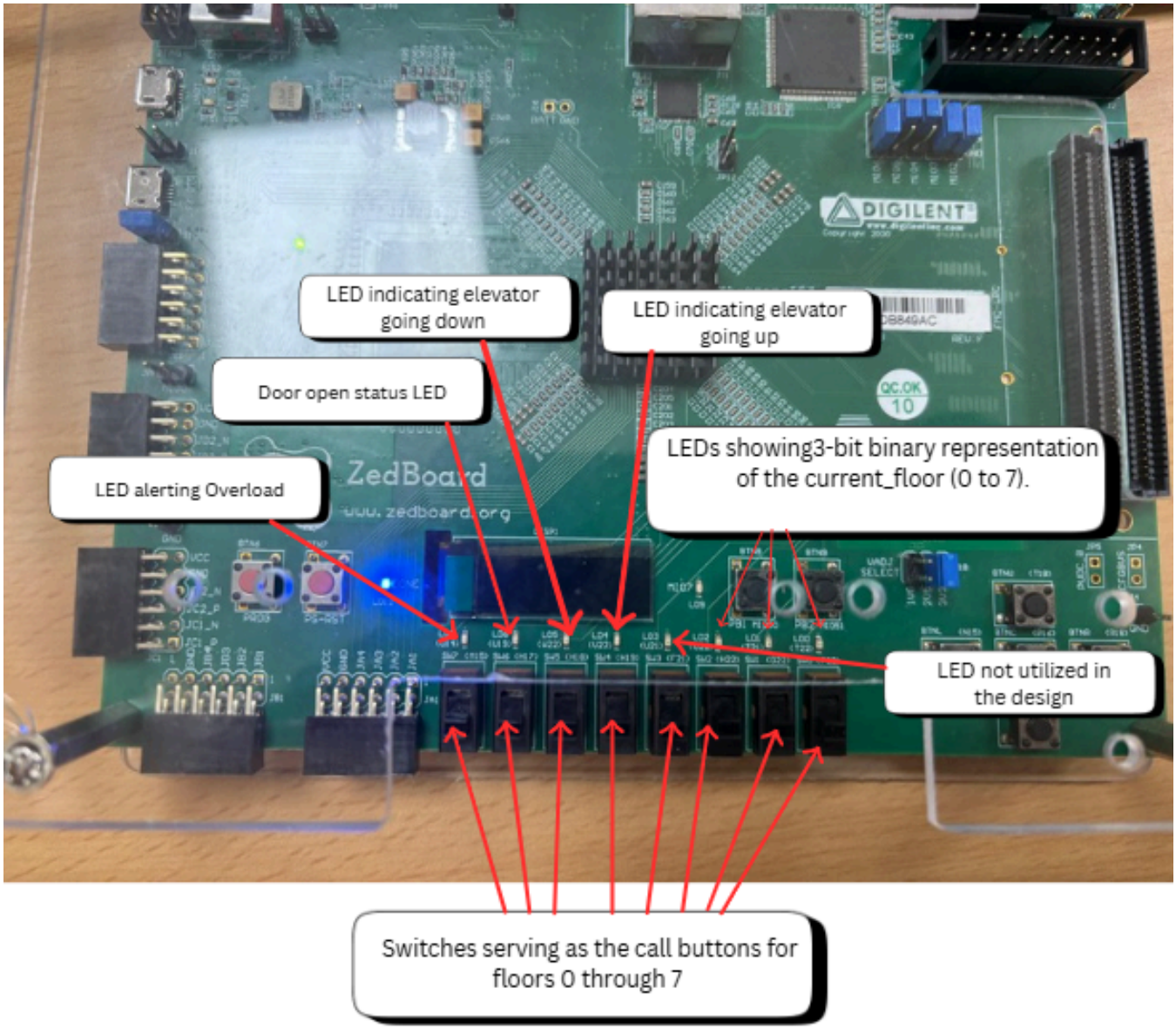
5.2.Complete Setup and Detailed Explanation of Experimental Setup

Explanation of Setup:

The experimental setup centers around the ZEDBoard, a widely used development platform featuring a Xilinx Zynq-7000 SoC.

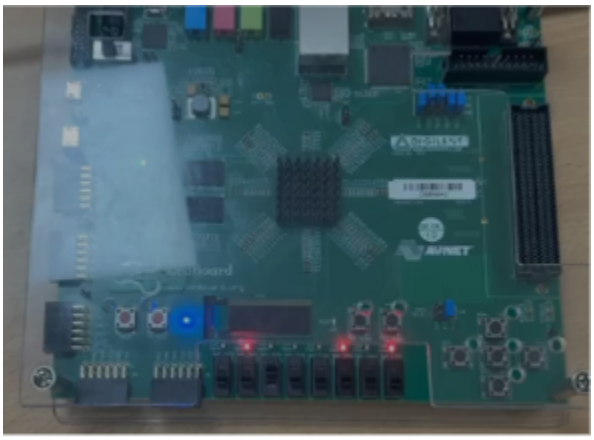
1. **Power and Programming:** The ZEDBoard is powered via its DC input and connected to the host PC via a USB cable, which handles both JTAG programming (for bitstream download) and serial communication (for optional debugging).
2. **Input Simulation:** The 8 slide switches on the board serve as the call buttons for floors 0 through 7. Because the design uses edge-detection, a user simulates a momentary call by flipping the switch OFF then ON again. The reset button is configured via the XDC file to trigger the global reset for the FSM and all synchronous registers.
3. **Output Visualization:** The 8 user LEDs provide immediate, real-time feedback on the elevator's status.
    - **LEDs [2:0] (Floor Position):** A 3-bit binary representation of the `current_floor` (0 to 7).
    - **LEDs [4:5] (Direction):** LEDs indicating `D_UP`, `D_DOWN`, or `D_IDLE`.
    - **LED [6] (Door Status):** High when the door is open (`door_open`).
    - **LED [7] (Overload):** Indicates the overload condition .

This setup allows a user to directly interact with the system by setting request switches and observing the deterministic movement, door cycling, and intelligent return-to-ground sequence, all operating at a human-perceptible, real-time rate of 1 Hz.

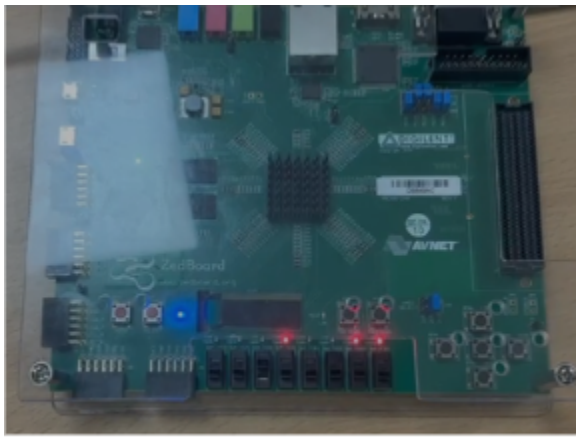

LED indicating elevator going down
LED indicating elevator going up
Door open status LED
LED alerting Overload
LEDs showing3-bit binary representation of the current_floor (0 to 7).
LED not utilized in the design
Switches serving as the call buttons for floors 0 through 7
ZedBoard
DIGILENT

6. Results

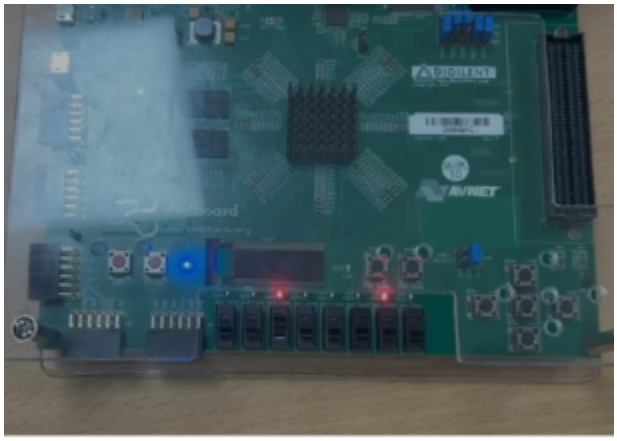
6.1 Photographs of real-time functional results



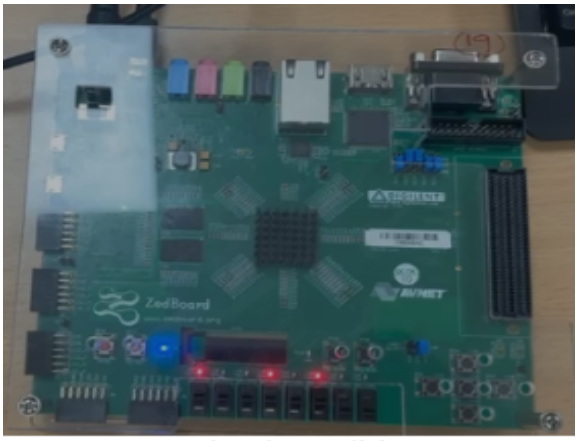
The elevator has reached the fifth floor and the doors are currently open



The elevator is on the 3rd floor but continues moving upward



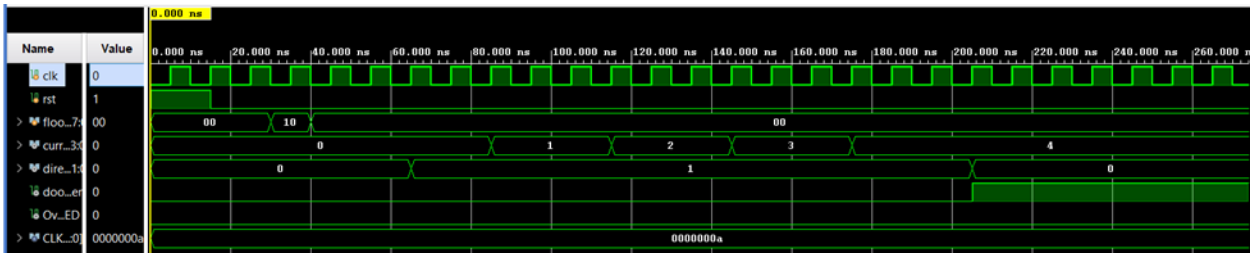
The elevator is on the 2nd floor but continues moving downward

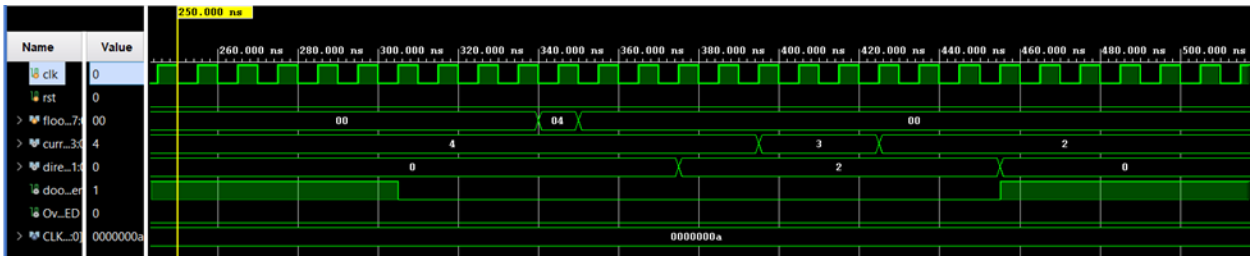


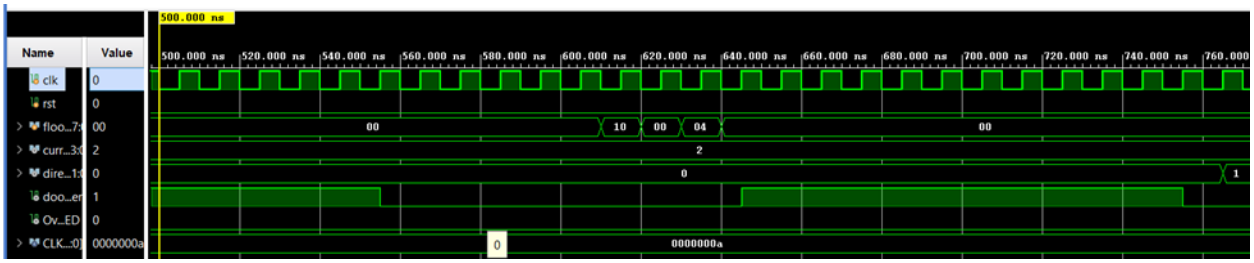
Overload Condition

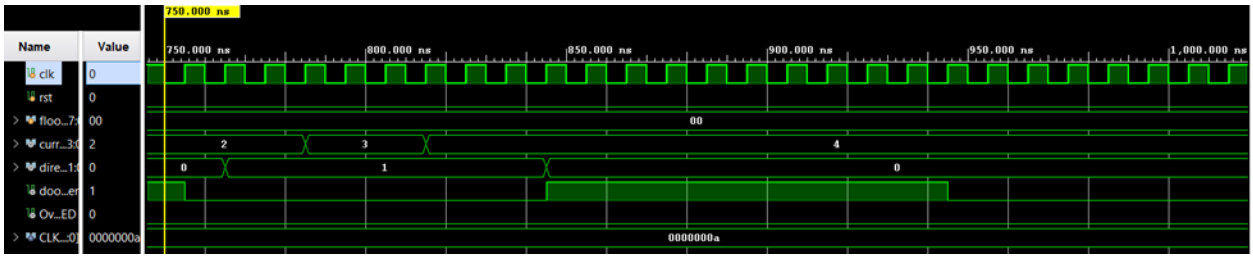
6.2 Post-synthesis/Post-route Simulation

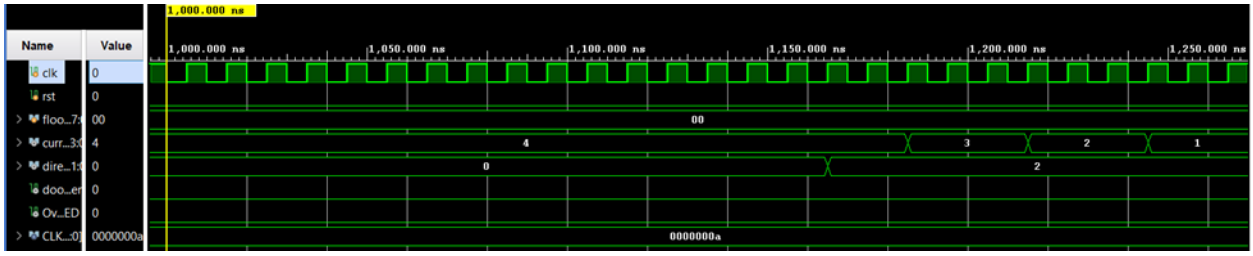
Simulations



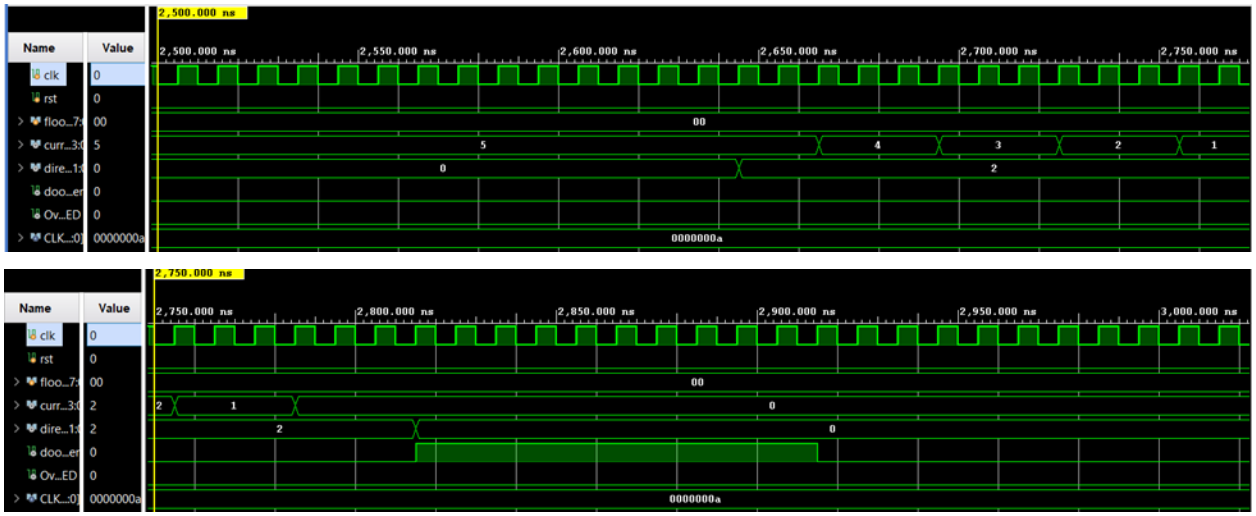




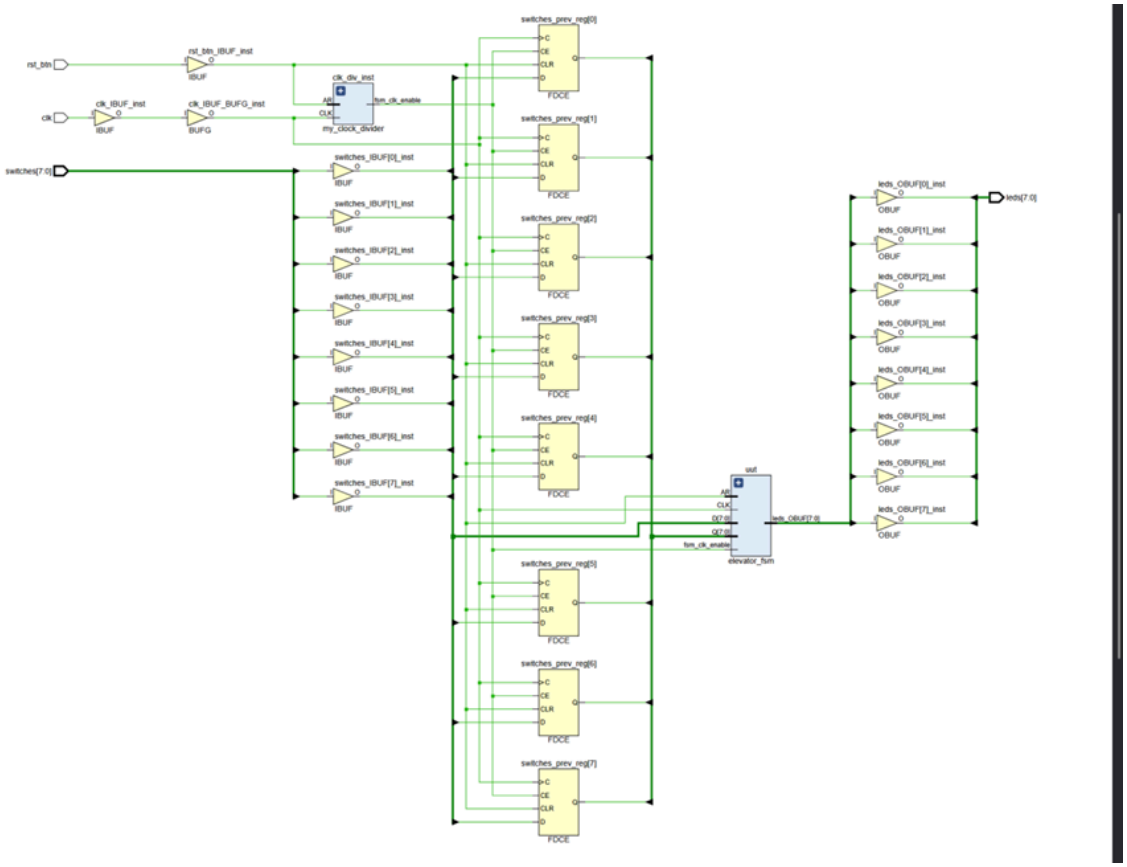




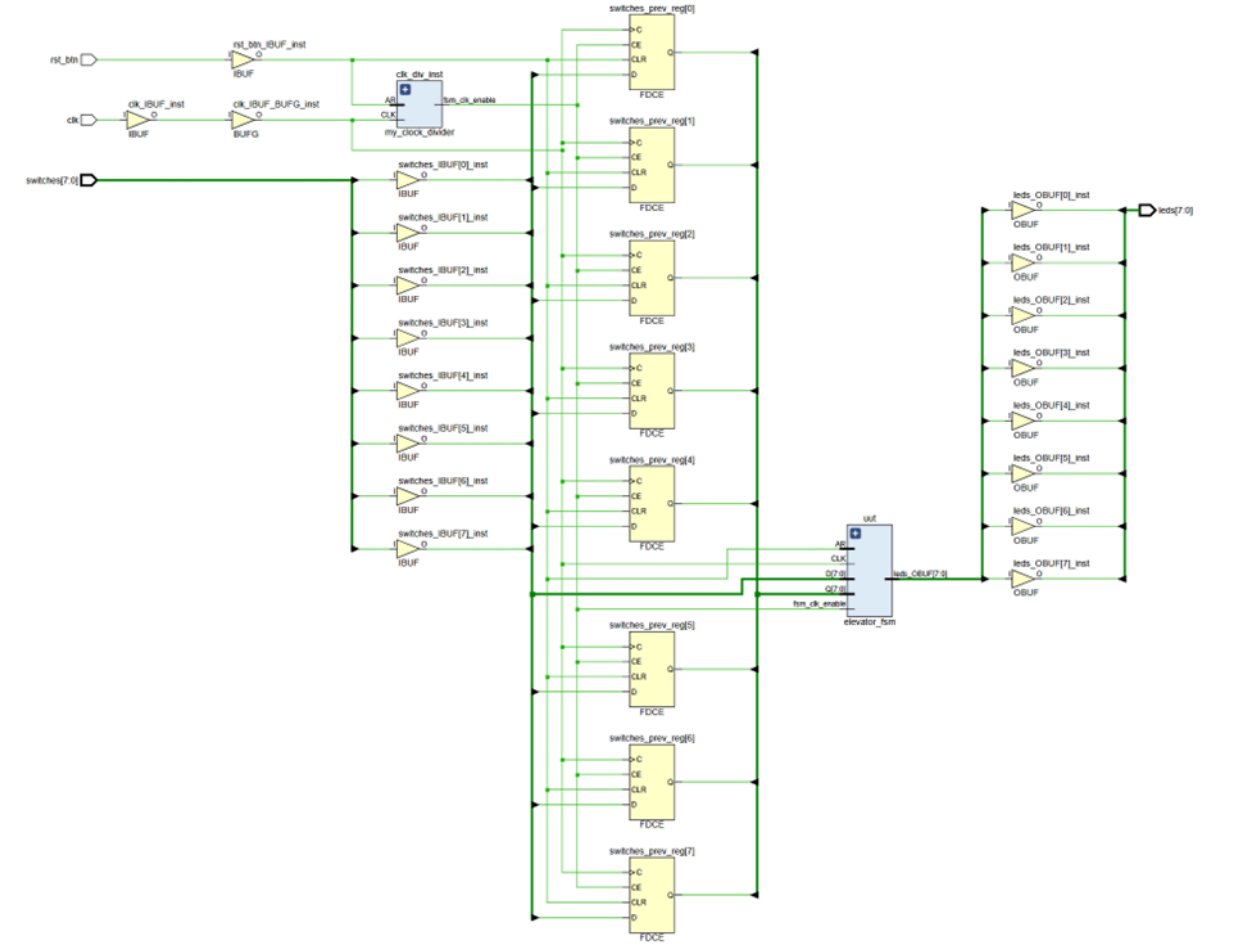

Name
Value
clk
rst
floo...7
curr...3
dire...1
doo...en
Ov...ED
CLK...0
1,250.000 ns
1,300.000 ns
1,350.000 ns
1,400.000 ns
1,450.000 ns
1,500.000 ns
1,500.000 ns
1,550.000 ns
1,600.000 ns
1,650.000 ns
1,700.000 ns
1,750.000 ns
1,750.000 ns
1,800.000 ns
1,850.000 ns
1,900.000 ns
1,950.000 ns
2,000.000 ns
2,000.000 ns
2,050.000 ns
2,100.000 ns
2,150.000 ns
2,200.000 ns
2,250.000 ns
2,250.000 ns
2,300.000 ns
2,350.000 ns
2,400.000 ns
2,450.000 ns
2,500.000 ns
0000000a



Post Synthesis:



Post Implementation:



6.3 Resource Utilisation, Timing, Power consumption

Utilization — **Post-Synthesis** | Post-Implementation

Graph | **Table**

Resource	Estimation	Available	Utilization %
LUT	124	53200	0.23
FF	102	106400	0.10
IO	18	200	9.00
BUFG	1	32	3.13

Utilization — Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	124	53200	0.23
FF	102	106400	0.10
IO	18	200	9.00
BUFG	1	32	3.13

Design Runs | Methodology | DRC | Power | Timing ×

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	inf	Worst Hold Slack (WHS):	inf	Worst Pulse Width Slack (WPWS):	NA
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	NA
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	NA
Total Number of Endpoints:	226	Total Number of Endpoints:	226	Total Number of Endpoints:	NA

There are no user specified timing constraints.

Reports | Design Runs | Methodology | DRC | Power × | Timing

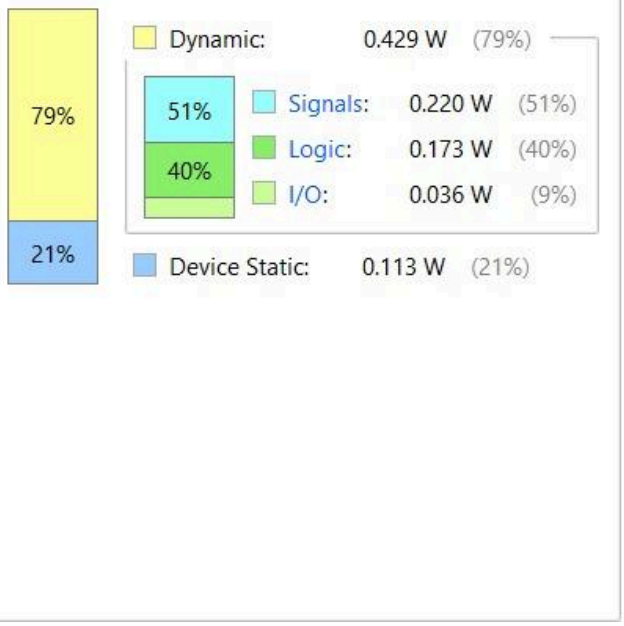
Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.542 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**31.2°C**
Thermal Margin:	53.8°C (4.5 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	11.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power



7. Conclusion

The FPGA-Based Intelligent Elevator Control System demonstrates a compact, deterministic, and testable hardware implementation of an 8-floor elevator controller. By partitioning the design into a top wrapper, a precision clock divider and a central FSM, the project helps achieve clear separation of concerns: reliable input synchronization (edge detection), predictable state-driven control core etc. The system meets the stated objectives for example, it services concurrent floor requests, enforces deterministic movement and door timings, provides visual feedback through the LEDs, and implements a simple safety indicator (overload LED).

Strengths -

- **Deterministic timing:** using a 100 MHz to 1 Hz clock enable ensures all timers and state updates map directly to seconds, simplifying verification and user interaction.
- **Reliable input handling:** edge detection converts persistent switch inputs into single-cycle pulses, preventing duplicate queue entries.
- **Modularity & testability:** hierarchical modules (top, clock_divider, elevator_fsm) made simulation, synthesis, and hardware validation straightforward.
- **Hardware realization:** the design was synthesized, simulated, and run on the ZEDBoard, verifying real-time behavior and LED diagnostics.

Recommendations -

- We can enhance scheduling (e.g., elevator grouping, look-ahead or scan algorithms) to reduce average wait time.
- We can replace time-only movement with sensor/encoder feedback to support precise stopping and smoother motion profiles.
- Add input debouncing and real push-button support, plus telemetry (UART/USB) for logging and interactive debugging.
- Formalize safety checks (door-interlock) and consider formal verification for critical control paths.
- Expand the UI (7-segment or VGA) for more informative status displays.

Overall, the project provides a robust, educational, and extensible hardware foundation for intelligent elevator control. It validates the advantages of FPGA implementation like parallelism,

determinism, and low-latency response, while offering several clear directions for functional and safety enhancements toward an industrially viable system.

8. References

1. FPGA prototyping by Verilog examples. By Pong P. Chu., Wiley, 2008
2. Xilinx Inc. *Vivado Design Suite User Guide: Synthesis (UG901)*.
3. Xilinx Inc. *Vivado Design Suite User Guide: Implementation (UG904)*.
4. Xilinx Inc. *7 Series FPGAs Configuration User Guide (UG470)*.
5. Xilinx Inc. *Zynq-7000 SoC Technical Reference Manual*.

9. Appendix

1. Verilog modules

Elevator FSM

```
`timescale 1ns / 1ps

module elevator_fsm (

        input clk,

        input clk_enable,

        input rst,

        input [7:0] floor_request,

        output reg [3:0] current_floor,

        output reg [1:0] direction,

        output reg door_open,

        output reg Overload_LED

);

        // --- Parameters ---

        parameter FLOOR_0 = 4'd0;

        parameter FLOOR_7 = 4'd7;
```

```
// State parameters
parameter S_IDLE      = 2'b00;
parameter S_MOVING_UP  = 2'b01;
parameter S_MOVING_DOWN = 2'b10;
parameter S_DOOR_OPEN  = 2'b11;

// Direction parameters
parameter D_IDLE = 2'b00;
parameter D_UP   = 2'b01;
parameter D_DOWN = 2'b10;

// Door timer
parameter DOOR_OPEN_CYCLES = 10;
parameter MOVE_CYCLES_PER_FLOOR = 2;
parameter QUEUE_DEPTH = 8;

parameter IDLE_TIMEOUT_CYCLES = 20;
parameter OVERload_led_cycles = 10;

// Internal Registers
reg [1:0] state, next_state;
reg [3:0] door_timer;
reg [1:0] move_timer;

reg [4:0] idle_timer;
reg [3:0] overload_timer;

// FIFO Queue Registers
reg [3:0] queue [0:QUEUE_DEPTH-1];
```

```
reg [3:0] head;

reg [3:0] tail;

reg [3:0] count;

reg [7:0] requests_to_enqueue;

reg [3:0] current_target_floor;

reg target_valid;


//  Wires for Queue & Encoder

wire queue_empty = (count == 0);

wire queue_full = (count == QUEUE_DEPTH);


// Priority Encoder

wire [2:0] pe_index;

wire       pe_valid;


// 8:3 priority encoder

assign pe_index = (requests_to_enqueue[0]) ? 3'd0 :

(requests_to_enqueue[1]) ? 3'd1 :

(requests_to_enqueue[2]) ? 3'd2 :

(requests_to_enqueue[3]) ? 3'd3 :

(requests_to_enqueue[4]) ? 3'd4 :

(requests_to_enqueue[5]) ? 3'd5 :

(requests_to_enqueue[6]) ? 3'd6 :

(requests_to_enqueue[7]) ? 3'd7 :

3'd0; //default case


assign pe_valid = |requests_to_enqueue;

reg [7:0] next_requests_to_enqueue;
```

```
// Combinational Logic (for next state and latch)
always @(*) begin
next_state = state;
direction = D_IDLE;
door_open = 0;

case (state)
S_IDLE: begin
direction = D_IDLE;
if (target_valid) begin
if (current_floor < current_target_floor) begin
                next_state = S_MOVING_UP;
end else if (current_floor > current_target_floor) begin
                next_state = S_MOVING_DOWN;
end else begin
                next_state = S_DOOR_OPEN;
end
end
end

S_MOVING_UP: begin
direction = D_UP;
if (move_timer == MOVE_CYCLES_PER_FLOOR) begin
if (current_floor == current_target_floor) begin
                next_state = S_DOOR_OPEN;
end
end
end
```

```
S_MOVING_DOWN: begin
direction = D_DOWN;
if (move_timer == MOVE_CYCLES_PER_FLOOR) begin
if (current_floor == current_target_floor) begin
                next_state = S_DOOR_OPEN;
end
end
end

S_DOOR_OPEN: begin
direction = D_IDLE;
door_open = 1;
if (door_timer == DOOR_OPEN_CYCLES) begin
next_state = S_IDLE;
end
end

default: begin
next_state = S_IDLE;
direction = D_IDLE;
door_open = 0;
end
endcase

// Latch & Clear Logic (Combinational)

next_requests_to_enqueue = requests_to_enqueue;
```

```
next_requests_to_enqueue = requests_to_enqueue | floor_request;

if (pe_valid && overload_timer == 0) begin

next_requests_to_enqueue = next_requests_to_enqueue & ~(1'b1 << pe_index);

end


else if (pe_valid && overload_timer > 0) begin

next_requests_to_enqueue = next_requests_to_enqueue & ~(1'b1 << pe_index);

end

end


// Sequential Logic  for State, Floor, Timers, Queue

always @(posedge clk or posedge rst) begin

if (rst) begin

// Reset all registers

state <= S_IDLE;

current_floor <= FLOOR_0;

door_timer <= 0;

move_timer <= 0;

idle_timer <= 0;

overload_timer <= 0;

Overload_LED <= 0;


// Reset queue

head <= 0;

tail <= 0;

count <= 0;

requests_to_enqueue <= 8'b0;

target_valid <= 0;

current_target_floor <= 0;
```

```
end else if (clk_enable) begin

state <= next_state;


requests_to_enqueue <= next_requests_to_enqueue;


if (overload_timer > 0) begin

overload_timer <= overload_timer - 1;

Overload_LED <= 1;

end else begin

Overload_LED <= 0;

end


// --- Queue Enqueue Logic ---

if (pe_valid && overload_timer == 0) begin

if (pe_index == current_floor) begin

if (state == S_IDLE) begin

                state <= S_DOOR_OPEN;

end else if (state == S_DOOR_OPEN) begin

                door_timer <= 0;

end


end else if (count < 5) begin

queue[tail] <= pe_index;

tail <= tail + 1;

count <= count + 1;


end

else begin
```

```
overload_timer <= OVERload_led_cycles;

Overload_LED <= 1;

end

end


//Target & Queue Dequeue Logic

if (state == S_IDLE && !target_valid) begin

if (!queue_empty) begin

current_target_floor <= queue[head];

target_valid <= 1;


end else if (idle_timer == IDLE_TIMEOUT_CYCLES && current_floor != FLOOR_0) begin

current_target_floor <= FLOOR_0;

target_valid <= 1;

end

end


if (next_state == S_DOOR_OPEN && state != S_DOOR_OPEN) begin

if (count > 0) begin

head <= head + 1;

count <= count - 1;

end


target_valid <= 0;

end


if (state == S_DOOR_OPEN) begin

if (!target_valid && !queue_empty) begin

current_target_floor <= queue[head];
```

```
            target_valid <= 1;

            end

            end



            if (next_state == S_MOVING_UP || next_state == S_MOVING_DOWN) begin

            if (move_timer < MOVE_CYCLES_PER_FLOOR) begin

            move_timer <= move_timer + 1;

            end else begin

            move_timer <= 0;

            end

            end else begin

            move_timer <= 0;

            end



            if ((state == S_MOVING_UP) && (next_state == S_MOVING_UP) && (move_timer == MOVE_CYCLES_PER_FLOOR)) begin

            if (current_floor < FLOOR_7) begin

            current_floor <= current_floor + 1;

            end

            end

            else if ((state == S_MOVING_DOWN) && (next_state == S_MOVING_DOWN) && (move_timer ==
MOVE_CYCLES_PER_FLOOR)) begin

            if (current_floor > FLOOR_0) begin

            current_floor <= current_floor - 1;

            end

            end



            // Door Timer Update

            if (next_state == S_DOOR_OPEN) begin

            if (door_timer < DOOR_OPEN_CYCLES) begin
```

```
            door_timer <= door_timer + 1;
            end
            end else begin
            door_timer <= 0;
            end

            //  Idle Timer Update
            if (state == S_IDLE && !target_valid && queue_empty) begin
            if (current_floor != FLOOR_0) begin
            if (idle_timer < IDLE_TIMEOUT_CYCLES) begin
                        idle_timer <= idle_timer + 1;
            end
            end else begin
            idle_timer <= 0;
            end
            end else begin
            idle_timer <= 0;
            end

            end
            end

endmodule
```

Clock Divider

```
            `timescale 1ns / 1ps
```

```
module my_clock_divider (
        input clk_in,
        input rst_in,
        output clk_out
);

        parameter COUNT_MAX = 100_000_000 - 1;
        reg [26:0] counter_reg;

        always @(posedge clk_in or posedge rst_in) begin
        if (rst_in) begin
        counter_reg <= 0;
        end else begin
        if (counter_reg == COUNT_MAX) begin
        counter_reg <= 0;
        end else begin
        counter_reg <= counter_reg + 1;
        end
        end
        end

        assign clk_out = (counter_reg == COUNT_MAX);

Endmodule
```

Top Module

```
`timescale 1ns / 1ps


module zedboard_elevator_top (
                input clk,
                input rst_btn,
                input [7:0] switches,
                output [7:0] leds
);


                wire fsm_clk_enable;


                my_clock_divider clk_div_inst (
                .clk_in(clk),
                .rst_in(rst_btn),
        .clk_out(fsm_clk_enable)
                );


                reg [7:0] switches_prev;
                wire [7:0] floor_request_pulse;


                always @(posedge clk or posedge rst_btn) begin
                if (rst_btn) begin
                switches_prev <= 8'b0;
                end else if (fsm_clk_enable) begin
                switches_prev <= switches;
                end
```

```
        end

        assign floor_request_pulse = (switches & ~switches_prev);


        wire [3:0] current_floor;

        wire [1:0] direction;

        wire door_open;

        wire Overload_LED;

        elevator_fsm uut (

        .clk(clk),

    .clk_enable(fsm_clk_enable),

        .rst(rst_btn),

    .floor_request(floor_request_pulse),

    .current_floor(current_floor),

    .direction(direction),

    .door_open(door_open),

    .Overload_LED(Overload_LED)

        );

        assign leds[3:0] = current_floor;

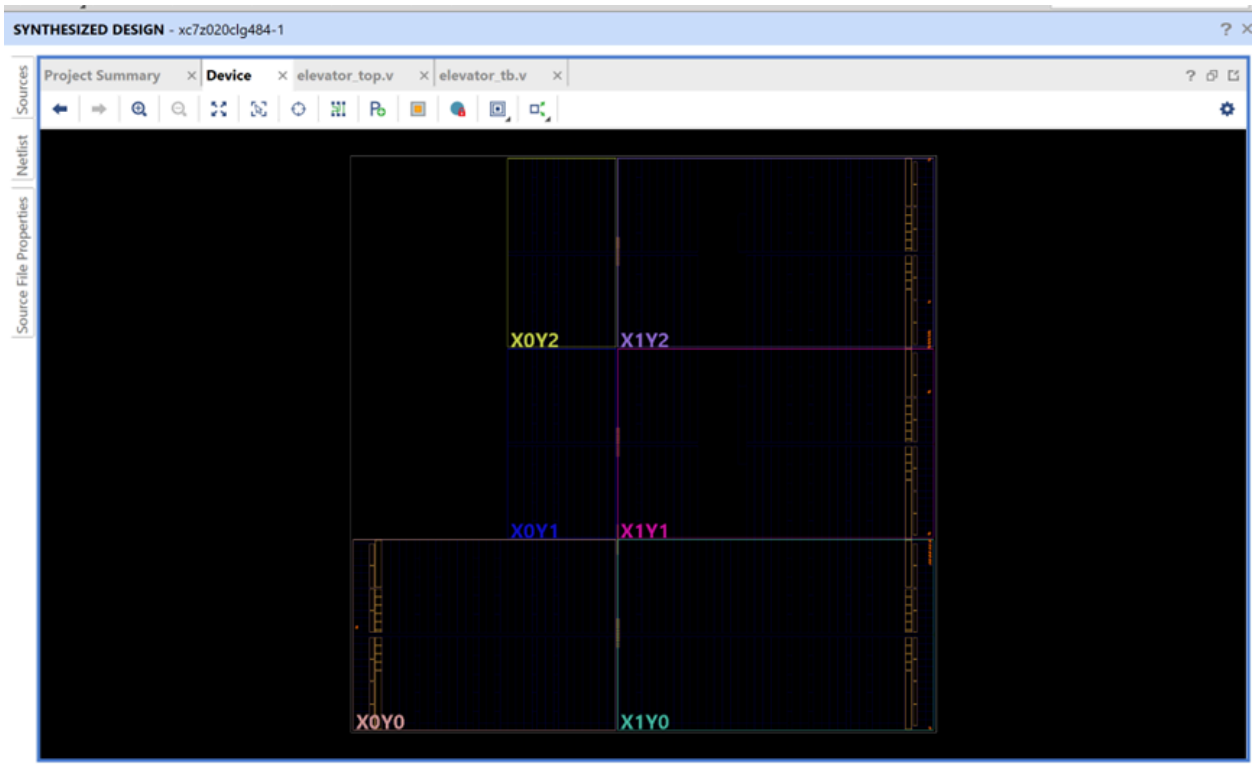
        assign leds[5:4] = direction;

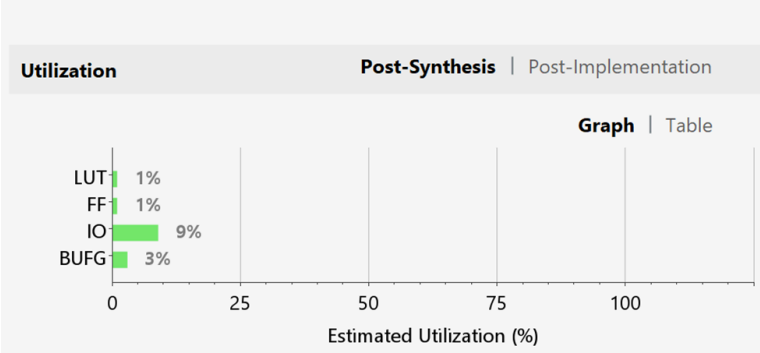
        assign leds[6]   = door_open;

        assign leds[7]   = Overload_LED;

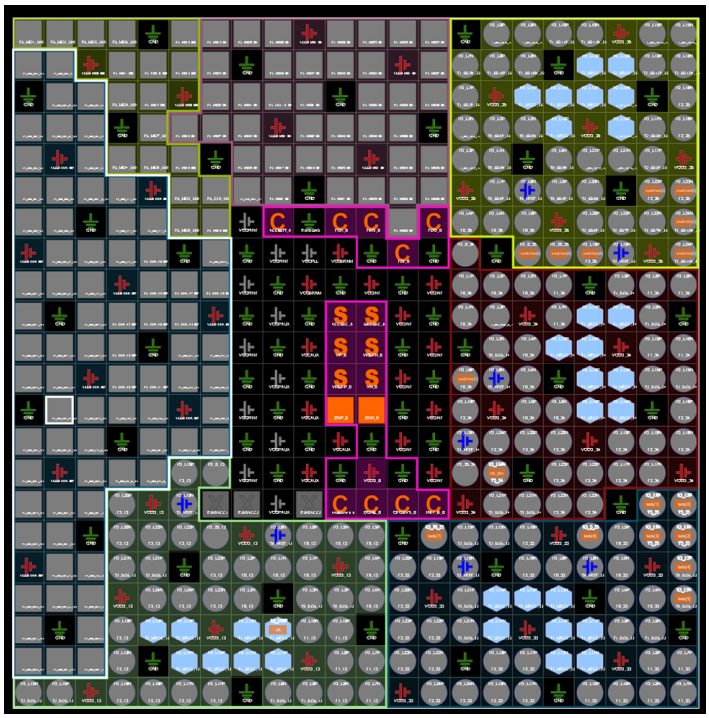
Endmodule
```

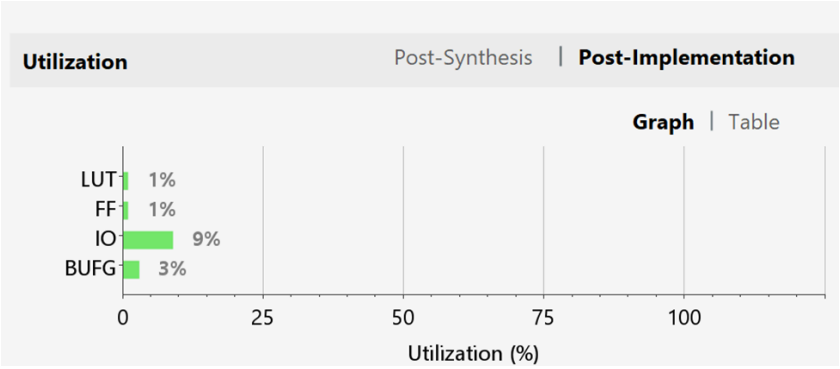
2. Synthesis


SYNTHESIZED DESIGN - xc7z020clg484-1
Project Summary
Device
elevator_top.v
elevator_tb.v
Sources
Netlist
Source File Properties
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0


Utilization
Post-Synthesis | Post-Implementation
Graph | Table
LUT 1%
FF 1%
IO 9%
BUFG 3%
0
25
50
75
100
Estimated Utilization (%)

3. Implementation





4. Real-Time Functional Testing

